Twelve Quick Tips for Managing IT Disasters in Small Research Software Teams

Greg Wilson[1*]

1 Third Bit, Toronto, Ontario, Canada
* Corresponding author, gvwilson@third-bit.com.

In 2025, the US government launched an unprecedented series of attacks on its own scientific research groups. A year later GitHub[1] dropped below 90% availability for the first time, while wildfires in Canada, France, Spain, and elsewhere forced researchers from the homes and labs. These events and others have reminded us just how fragile research computing systems can be, and that planning for disasters is one of the most effective ways to prevent them.

This paper is a short guide to disaster planning and recovery for a small research software team. The tips assume that you *aren't* an experienced system administrator, but are willing to do some things on top of your regular job. Where expertise is required, most research institutions have research computing groups, data librarians, and environmental health-and-safety offices whose entire job is to help with exactly these problems. The "Questions for Support" boxes throughout tell you what to ask them for; this paper tells you what "done" looks like. (If you don't have such a support team, start with the core described below and try out the free-tier tools named in the tips if and when you have time.)

If you only adopt a handful of these tips, the essential core is: make a risk list and a written plan (Tips 1-2), follow the 3-2-1 backup rule (Tip 3), turn on multi-factor authentication with a shared password manager (Tip 7), choose one out-of-band way to stay in touch (Tip 4), and set up basic monitoring (Tip 6). The rest can be delegated to the specialists named above or deferred until you need them.

Who You Are

We assume that your team fits one or more of these profiles:

Providing a simple online service Your research group runs a Shiny[2] app or a Streamlit[3] dashboard so that collaborators can query data. Disasters scenarios include:

- Your cloud provider suspends your account.
- The graduate student who deployed the service leaves and no one else has credentials.
- Your service depends on a database that another group maintains, and that group shuts down on short notice.

Publishing a software package Your group releases software to public registries like PyPI[4] or CRAN[5]. Scenarios include:

[1] https://github.com/
[2] https://shiny.posit.co/
[3] https://streamlit.io/
[4] https://pypi.org/
[5] https://cran.r-project.org/

- Your PyPI token expires and the person who created it has left.
- GitHub suspends your account.
- Your institutional web hosting is turned off.

Note that once a package is published, the registry copy itself is usually not a single point of failure (CRAN, for example, has multiple mirrors). The risks are the accounts and credentials around the release.

Managing data Your group produces datasets in the lab or in the field that are shared with collaborators and deposited in repositories. Scenarios include:

- A lab server's hard drive fails.
- A field laptop is stolen and the data on it has not been backed up.
- Your data repository announces it is shutting down.

These categories overlap: for example, an ecology lab may publish an R package, run a Shiny dashboard for collaborators, and curate a ten-year field dataset. At the other extreme, a team of one or two people who only write analysis code and publish a package may find all twelve tips disproportionate; such teams should focus on the essential core named in the introduction and ignore, defer, or delegate the rest.

Tip 1: Know your risks.

Start by making a point-form list of the services, data sets, and physical assets your team depends on. For a small team, this will take about an hour the first time through, and 15-30 minutes per quarter for review. For our ecology lab, this includes where the R package is published, where the Shiny dashboard is hosted, where the ten-year field dataset lives, and the laptops the team works on. For each item, answer two questions:

1. How long can you be without it before work stops dead? This is your *Recovery Time Objective* (RTO).
2. How much data can you afford to lose: the last hour's work, the last day's, a week's? This is your *Recovery Point Objective* (RPO).

A Google Doc or a text file in a shared drive or Git repository is the right tool for creating this, but as discussed below, make sure you keep a copy somewhere you can access when online services are down. Once you have this list, identify the single points of failure within your control, such as the person who knows the deployment process or the credit card that pays for cloud services. For each, ask what happens if it disappears.

Questions for Support

If you have access to research support teams of the kind described in the introduction, you may want to ask them these questions.

- Can you help us estimate realistic recovery time and recovery point targets?
- Who else shares the systems we depend on, and how would their failure cascade to us?

Tip 2: Make a plan.

Once you understand your risks, write a plan in a single shared document that everyone on the team can find in thirty seconds or less. Store the plan in at least two places that cannot fail simultaneously, such as the shared team drive *plus* a printed copy in a desk drawer or a PDF saved on every team member's phone. Pin the link in your team chat, and if you have a physical office, tape a printed copy to the fridge.

State the disaster declaration criteria in plain language, such as "our dashboard has been offline for more than a day" or "we have found a virus on a server". Every team member is part of the recovery team, so make sure everyone has read the plan, understands the criteria, and knows their first action if a disaster is declared.

Your plan should include a short checklist for each scenario, written as a numbered series of specific actions. For example, for an online service outage:

1. Send a message to the Signal[6] group.
2. Log into the database management console and click "Restore from snapshot".
3. Check that the dashboard loads and returns data correctly.

Note

If a step requires knowledge or access that only one person has, you have a *lottery-factor* problem (also called a "bus factor" problem). Ask yourself, "If Alice wins the lottery and moves to Bali tomorrow, can the rest of the team keep things running?" This is the hardest part of planning: Alice probably doesn't recognize all the things only she knows how to do, and no one else does either because she has always taken care of it.

Questions for Support

- Will you review our plan and checklists and flag any steps that are wrong or missing?
- Where should we keep a copy so your staff can also find it during an incident?
- Who is our first point of contact on your team when we declare a disaster?

Tip 3: Back up everything.

Follow the *3-2-1 rule* [1]: three copies of every critical asset, on two different types of media, with one copy off-site. This applies to data, software releases, electronic lab notebooks, and configuration information. For our ecology lab, the three copies might be:

1. The working copy on a lab server or shared drive.
2. An automated backup. If your institution offers a research computing backup service, use it. If not, use a managed database (e.g., Amazon RDS[7] or Google Cloud SQL[8]) with automatic daily snapshots (no scripting required). For files, GUI tools like Duplicati[9] or your cloud provider's desktop sync app are sufficient.

[6] `https://signal.org/`
[7] `https://aws.amazon.com/rds/`
[8] `https://cloud.google.com/sql`
[9] `https://duplicati.com/`

3. An off-site copy. For raw data, keep a second backup in a different location or cloud account, separate from your working copy and your automated backup. Publishing is a related but distinct step: deposit the *curated* version of a dataset in an institutional or domain repository (Dataverse[10], Dryad[11], Zenodo[12], or OSF[13]) to get a citable DOI. For software, connect Zenodo or Figshare[14] to your repository so every tagged release is automatically archived. For lab notebooks, scan or photograph key pages quarterly and save them with your other backups.

Back up your source code to at least two different software forges (e.g., GitLab[15] as well as GitHub) and turn on automatic mirroring. A `git push` to a secondary remote is the most cost-effective one-liner you will ever write, but automatic mirroring is even better.

Back up your configuration as well, such as DNS records, environment variables, and pipeline definitions. These are small in size but catastrophic to reconstruct from memory. A single export once a week, stored alongside your other backups, is probably sufficient. Store backups under a separate account whose credentials are not kept on any production server, so a compromise of your main account does not also reach your backups.

Finally, *test a full restore from backup* at least once a year: according to one recent report, almost 20% of backups fail to restore completely [2]. And remember that you may need to restore several times in a real disaster, so check that your restore process is *idempotent*, i.e., that you can run it and then run it again. A recovery that corrupts data the second time is as bad as no recovery at all.

Questions for Support

- What backup and off-site repository services do you provide?
- Can you test our restore process?
- Who manages our off-site deposits, and how do we reach them when we need a restore?

Tip 4: Decide how to stay in touch.

Choose a single communication channel that does not depend on your normal infrastructure. For example, if you use Slack[16], agree *in advance* on a Signal group or a phone tree for when Slack is unavailable. Make sure the fallback is written in your plan.

For lab and field staff, maintain a printed contact tree: if the server fails at 2:00 a.m., who calls whom? Post it on the lab fridge and save a photo on every team member's phone.

Keep a printed (or phone-screenshot) contact list with every team member's mobile number and a personal email address. Do not store this only in a shared drive that may be affected by an outage.

Pre-write messages and store them with the plan and in your password manager's shared vault:

[10] https://dataverse.org/
[11] https://datadryad.org/
[12] https://zenodo.org/
[13] https://osf.io/
[14] https://figshare.com/
[15] https://gitlab.com/
[16] https://slack.com/

1. For the team: "This is a declared incident: meet in the Signal group."
2. For users of your online service: "We are investigating an outage and will update you within 60 minutes."
3. For collaborators waiting on data: "We have lost connectivity temporarily. Data collected so far is safe; expect a delay of [X] days."
4. For afterward: "Service is restored and the situation is under control. Here is what happened and what we are doing to prevent recurrence."

Designate one person as the communicator so that everyone else can focus on fixing the problem. The communicator does not need to be technical: they need to be calm and reliable. Silence erodes trust faster than bad news, so update stakeholders at regular intervals (for example, every 60 minutes or at 10:00 am and 3:00 pm) even if the update is "still working on it".

Questions for Support

- How can we reach your team during an outage?
- Is there an institutional status page or mailing list our updates should be mirrored to?
- Who on your team needs to receive our incident messages?

Tip 5: Test the plan.

The most revealing test for a small team is to hand the recovery checklist to the newest team member, give them a test copy that they can break without consequences, and ask them to execute the plan without help. Every point where they get stuck is a documentation or automation gap.

If no one has joined your team recently, gather everyone for an hour-long walkthrough. Pick the thing that scares you most as "It's Tuesday morning and all of Monday's data has disappeared from the database." Walk through the checklist step by step and fix anything that is missing, wrong, or stale then and there.

Keep track of how long these tests take and compare against your RTO. If restoring the database takes four hours but your RTO is two hours, you must either fix the former or adjust the latter.

Questions for Support

- Can your staff join our next drill so we can find gaps together?
- Can you provide a sandbox where we can practice for disasters without affecting production?

Tip 6: Watch for trouble.

Monitor the things that would get you out of bed at 3:00 a.m. Those are the items on the risk list you made in Tip 1, especially the ones with the shortest RTO and RPO.

For online services, free-tier monitoring like Uptime Robot[17] and Healthchecks.io[18] require no code and are infinitely better than nothing.

Check that your dataset's DOI still resolves and that your package is still installable from its registry. If your software has a "downloads per month" badge, a sudden drop to zero may be the first sign that your registry listing is broken.

Check your cloud bill at least once a month: a misconfigured resource can generate a catastrophic bill even for a small team. Set budget alerts in your cloud provider's console at 50% and 90% of your normal monthly spend, sent to at least two people. Most cloud providers offer free-tier budget alerting adequate for a small team.

Questions for Support

- What monitoring and alerts do you already run, and can we subscribe to them?
- Can you help us set up alerts and uptime checks for our accounts?
- Which early-warning metrics would tell us something is about to fail?

Tip 7: Secure your accounts.

Enabling multi-factor authentication (MFA) on every account is the highest-impact security measure you can take [3]. An authenticator app or passkey is better than SMS-based MFA, though SMS is better than no MFA at all.

Use a password manager like 1Password[19] or Bitwarden[20] for the whole team so that you can share credentials securely. Every shared credential goes in the password manager: never send credentials over chat or email. Print the password manager's recovery code and store several copies somewhere physically safe: if you lose access to the password manager, you lose access to everything.

If you have a custom domain, know who manages it: institutional IT, a commercial registrar, or someone on the team. Make sure that person's contact information is in the disaster plan, that auto-renewal is enabled, and that at least two people can access the account. Set a quarterly calendar reminder to verify that auto-renewal is still working.

Make sure at least two people have critical administrative credentials, such as the cloud root account, the domain registrar login, the payment method for infrastructure, and the password-manager administrator account. These credentials should not be tied to a single phone number or email address. For physical access, maintain a list of who has keys to the lab, who knows the alarm code, and who is authorized to move hardware.

Finally, maintain an onboarding checklist alongside an offboarding one. When someone joins, grant their access, add them to the password manager and the contact tree, and have them read the plan and run the newest-member drill from Tip 5. When someone leaves, revoke their digital access and collect their physical keys as soon as you can. A disgruntled former team member with lingering access is trouble you don't need.

[17] https://uptimerobot.com/
[18] https://healthchecks.io/
[19] https://1password.com/
[20] https://bitwarden.com/

Questions for Support

- Can you help us set up multi-factor authentication?
- Which of your team should have access to administrative credentials?
- How quickly does your offboarding process revoke access when someone leaves?

Tip 8: Protect your assets.

Make a simple, explicit policy about where things live, such as, "patient data is never copied to a personal laptop" or "database dumps are encrypted at rest". If your policy will not fit comfortably on a single page in an 11-point font, rewrite it until it does.

Note

You may have obligations you are not aware of under HIPAA, GDPR, or your funder's data-management plan. If you are handling sensitive data, you should therefore check whether your institution has an ethics board or data-protection office that offers free consultation.

If your institution's IT group owns your servers and laptops, several of the steps below become things to verify with or request from them rather than do yourself; the "Questions for Support" box at the end lists the ones to raise.

Keep laptops and servers patched: enable automatic updates for operating systems, browsers, and any server-side software. The inconvenience of an unexpected reboot is less than the inconvenience of ransomware.

For on-premises equipment like a lab server, consider buying a cheap uninterruptible power supply (UPS) and configure it to trigger a clean shutdown when the battery reaches 20%. Replace the UPS battery every three years: they have a tendency to degrade silently.

Prepare a short, written response for ransomware and similar breaches. If someone on the team is contacted by villains, their first action should be to physically disconnect the affected machine from the network and then call the designated incident lead. Do not try to negotiate or pay without consulting legal counsel: most jurisdictions have regulations about ransomware payments.

Questions for Support

- Who should we talk to about data protection?
- Can you confirm our laptops and servers have automatic patching enabled?
- If we face ransomware, who is our incident lead and which legal counsel do we call?

Tip 9: Stabilize, then investigate.

The most important rule of incident response is, "Stabilize first, investigate later". When something *does* happen, assign clear roles for the duration of the incident: one

person leads the technical response, one person handles communication using the pre-written messages from Tip 4, and everyone else does what the leads tell them to.

Create an incident log *as you work*: a shared Google Doc or a thread in your out-of-band Signal group is good enough. This log keeps the team in sync and gives you a record for the post-incident review. Timestamp every significant action:

- 14:32: server outage alarm triggered
- 14:35: verified that all database requests are returning 404
- 14:40: began transition to backup server

Escalate early. If you are on a cloud provider's support plan, open a ticket the moment you suspect the problem is on their side. You can cancel your request if you fix the problem yourself, but you cannot get back the two hours you spent wrestling with something outside your control.

Finally, conduct a short review within 48 hours of any incident. The goal is to identify what in the system allowed the incident to happen, not who made a mistake. Write down one concrete action item and assign it to one person with a deadline. If you cannot find an outside moderator for the review, have the team member *least* involved in the incident lead it.

Questions for Support

- How do we escalate a ticket with our cloud provider(s)?
- Who on your team should join our incident response?
- Can you act as moderator for our post-incident review?

Tip 10: Count the cost.

You probably don't have a budget line item titled "disasters", but you still have costs. Digital costs include cloud backup storage, password-manager subscription, domain and certificate renewals, managed-database fees, and repository deposit charges. More important, but harder to track, are the personnel costs: time spent on quarterly plan reviews, annual backup-restore tests and disaster drills, cross-training to reduce lottery factors, and the hours lost to incident response instead of research.

Calculate what a day of downtime costs you, including things like lost experiment time, missed paper deadlines, or collaborators who stop trusting you. That number tells you whether it is worth spending $50/month on a managed database or $200/year on a backup service.

Questions for Support

- Can you tell us what backup storage, a managed database, and monitoring ought to cost?
- Which of those can the institution cover, and which do we have to fund ourselves?

Tip 11: Support your team during the crisis.

The first ten tips are about systems and processes; these last two are about people, because a disaster is as much a human event as a technical one. Team members may be scared, angry, or ashamed, particularly if they believe they caused the problem. Handling the human damage is therefore just as important as handling the technical aspects.

During the acute phase of a disaster, apply the same five principles that the World Health Organization recommends for crisis responders [4]:

Promote safety. Before asking for status updates, confirm that everyone is not in immediate personal danger. Someone whose laptop was stolen may not be thinking clearly about data loss because they are still shaken from the theft itself.

Calm the environment. Do not add to the panic. Use a steady voice in your out-of-band chat. Remind the team that the plan exists and that they have rehearsed for this. Panic is contagious, but so is calm.

Build self-efficacy. Remind people that they have handled difficult things before. Give each person one concrete task from the checklist so that everyone feels useful rather than helpless.

Foster connectedness. Use your fallback communication channel to keep the team together. Isolation amplifies distress, so do not let anyone absorb the news alone.

Maintain hope. In particular, do not speculate about worst cases before you have confirmed them.

Protect the person who may have caused the problem. A team member who accidentally dropped a table from a database or clicked a phishing link probably feels worse than anyone else. Assign them a recovery task, even a small one, so they can contribute to the fix rather than sitting in the corner imagining their career is over. Blame can wait; recovery cannot.

Watch for exhaustion. Disaster response is an adrenaline sprint followed by a long slog. Two people working four hours each will make fewer mistakes than one person working eight hours straight, so rotate people out of the incident response when the acute phase is over, even if they insist they are fine. In particular, the person who pulled the all-nighter to restore the database should not also be the person writing the post-mortem the next morning.

Questions for Support

- What wellbeing or crisis support do you offer, and how do we access it?
- Are there any provisions for post-incident leave?
- How should crisis preparation and response be factored into annual performance reviews?

Tip 12: Help your team recover after the disaster.

In Tip 9 said to hold the review within 48 hours. That is for gathering facts while memory is fresh; a second, more reflective conversation a week or two later gives people time to process what happened emotionally. The goal of this second conversation is not to find root causes but to ask: “How are we doing?”

If someone on the team caused the disaster, handle it carefully, and make sure the rest of the team acts that way. Ask them privately if they are OK. If you find yourself wanting to revisit the mistake in every team meeting, you are the one who hasn't moved on.

Recognize that people recover at different speeds. One team member may be joking about it by Tuesday while another is still waking up anxious on Friday. Both responses are normal. Check in with each person individually a week after the incident, and again a month later. Again, the question is simply: "How are you doing with what happened?"

Watch for survivors' guilt. For example, if your lab's data survived a hacking intrusion but a neighbouring group lost everything, your team may not want to celebrate their own recovery when others are still suffering. Let them talk about it. If possible, offer practical help to affected colleagues, such as sharing backup space or simply bringing them coffee.

Finally, celebrate the recovery, not just the disaster. A short thank-you message that says "we handled this" costs nothing and pays dividends the next time something goes wrong.

Questions for Support

- What longer-term counselling or support can you offer staff in the weeks after the incident?
- Who else in the institution should we inform so they can support our recovery?
- Can you help us capture lessons learned so other teams benefit from what happened?

References

1. Krogh P. The DAM Book. O'Reilly; 2009. The source of the "3-2-1 rule" for backups.
2. Backblaze. 2024 State of the Backup: Survey Says Security Incidents and Data Loss on the Rise; 2024. Reports that approximately 20% of backups fail to restore completely. Available from: `https://www.backblaze.com/blog/2024-state-of-the-backup-security-incidents-and-data-loss-on-the-rise/`.
3. Smalls D, Wilson G. Ten quick tips for staying safe online. PLOS Computational Biology. 2021 3;17(3):e1008563. Available from: `https://doi.org/10.1371/journal.pcbi.1008563`. doi:10.1371/journal.pcbi.1008563.
4. Organization WH. Psychological first aid: Facilitator's manual for orienting field workers; 2013. How to manage a team's mental health during a crisis. Available from: `https://www.who.int/publications/i/item/psychological-first-aid`.
5. Australian Cyber Security Centre. Essential Eight Maturity Model; 2023. Pragmatic security controls that don't require an enterprise budget. Available from: `https://www.cyber.gov.au/business-government/asds-cyber-security-frameworks/essential-eight/essential-eight-maturity-model`.

6. Beyer B, Jones C, Petoff J, Murphy NR. Site Reliability Engineering. O'Reilly; 2016. Chapters 6 (monitoring), 8 (release engineering), and 15 (postmortem culture) are most applicable to small teams.

7. Cybersecurity and Infrastructure Security Agency. #StopRansomware Guide; 2023. Practical ransomware guidance applicable at any scale. Available from: https://www.cisa.gov/stopransomware/ransomware-guide.

8. National Institute of Standards and Technology. Contingency Planning Guide for Federal Information Systems. National Institute of Standards and Technology; 2010. 800-34 Rev. 1. Enterprise-scale, but the BIA and RTO/RPO concepts scale down. Available from: https://csrc.nist.gov/pubs/sp/800/34/r1/upd1/final.

9. Shortridge K. Security Chaos Engineering. O'Reilly; 2023. Probably more than you need, but you won't know 'til you read it. Available from: https://www.securitychaoseng.com/.

10. Tamburri DA, Blincoe K, Palomba F, Kazman R. "The Canary in the Coal Mine…" A cautionary tale from the decline of SourceForge. Software: Practice and Experience. 2020 7;50(10):1930-51. doi:10.1002/spe.2874.